# *Brute Force Solution to the Subset Sum Problem Using C++*

---

Ghala Alghamdi S23108212 - Hiba Amanulla S23108227

---

## 1. Introduction

The Subset Sum Problem is a fundamental computational problem frequently encountered in algorithm design, combinatorics, optimization, cryptography, and decision theory. It involves determining whether a subset of a given set of integers can be selected such that the sum of the chosen elements equals a specified target value. Because it belongs to the class of NP-complete problems, there is no known polynomial-time algorithm that solves all instances efficiently. Nevertheless, the problem is widely studied due to its theoretical importance and practical relevance.

In this project, the brute force method is implemented as the baseline solution for Subset Sum. The brute force approach systematically enumerates every possible subset of the input set and evaluates its sum. Although computationally expensive, brute force is conceptually straightforward, guarantees correctness through exhaustive enumeration, and illustrates the behavior of exponential-time algorithms. Implementing this method in C++ also provides insight into bitwise operations, subset generation techniques, and performance considerations for combinatorial search problems.

## 2. Problem Definition

Formally, the Subset Sum Problem is defined as follows.

Given:

- A finite set of integers:
  $\mathbf{S = \{ s_1, s_2, s_3, \ldots, s_{\square} \}}$
- A target integer:
  $\mathbf{T} \in \mathbb{Z}$

Objective:

Determine whether there exists a subset $\mathbf{S'} \subseteq \mathbf{S}$ such that:

$\sum (x \in S') \, x = T$

If such a subset exists, the algorithm should return it. If no valid subset exists, the algorithm must explicitly indicate that no solution is found. This represents the decision version of the problem, but returning an actual subset also addresses the constructive version.

# 3. Brute Force Approach Overview

A set of **n** elements has exactly:
$2^n$ possible subsets.

(This is because each element has two independent choices: include it or exclude it.)

For example, if **n = 5**, then:
$2^5 = 32$ subsets must be evaluated.

### Core steps of the algorithm:

1. **Subset Generation:**
   Each subset is represented by a bitmask, where the i-th bit indicates whether the i-th element is included.

2. **Subset Evaluation:**
   Compute the sum of all selected elements.

3. **Target Comparison:**
   If the sum equals **T**, return the corresponding subset.

4. **Termination:**
   Stop when a matching subset is found or when all $2^n$ subsets have been checked.

Although this approach guarantees correctness, the number of subsets grows exponentially with **n**, making brute force impractical for larger inputs.

# 4. Pseudocode

```
FUNCTION SubsetSumBruteForce(nums, target):

    n ← length(nums)
    totalSubsets ← 2^n

    FOR mask FROM 0 TO totalSubsets - 1 DO:
```

```
        subset ← empty list
        sum ← 0

        FOR i FROM 0 TO n - 1 DO:
            IF (mask AND (1 << i)) ≠ 0 THEN
                add nums[i] to subset
                sum ← sum + nums[i]
            ENDIF
        END FOR

        IF sum = target THEN
            RETURN subset
        ENDIF

    END FOR

    RETURN empty list
```

This pseudocode uses bit-level operations to determine element inclusion.

# 5. C++ Code

```cpp
#include <iostream>
#include <vector>
using namespace std;

/*
   subsetSumBruteForce
   -------------------
   Uses brute force with bitmasking to examine every possible subset
   of the input list. For each subset, the function calculates the sum
   of the included elements and checks whether it matches the target.

   Important:
   - Subsets are examined in increasing bitmask order (0 to 2^n - 1).
   - Because of this, the function returns the *first* subset that
     matches the target sum.
   - If no subset matches the target, the function returns an empty vector.

   Returns:
       A vector containing the first valid subset that sums to the target,
       or an empty vector if no such subset exists.
*/
vector<int> subsetSumBruteForce(const vector<int>& nums, int target) {
```

```
    int n = nums.size();
    int totalSubsets = 1 << n; // 2^n possible subsets

    // Loop through all possible subsets
    for (int mask = 0; mask < totalSubsets; mask++) {

        vector<int> subset;
        int sum = 0;

        // Check each bit to determine which elements to include
        for (int i = 0; i < n; i++) {
            if (mask & (1 << i)) {   // If bit i is 1 -> include nums[i]
                subset.push_back(nums[i]);
                sum += nums[i];
            }
        }

        // If this subset matches the target, return it
        if (sum == target) {
            return subset;
        }
    }

    // If no subset equals the target, return empty vector
    return {};
}

int main() {

    int n, target;

    cout << "Enter number of elements: ";
    cin >> n;

    // Hard stop: exit if n is greater than 30
    if (n > 30) {
        cout << "\nERROR: Brute force subset search cannot handle more than 30
elements." << endl;
        cout << "2^n grows too fast, making computation infeasible.\n";
        cout << "Please restart the program and enter 30 or fewer elements.\n";
```

```
        return 0;   // Exit program immediately
    }

    vector<int> nums(n);
    cout << "Enter " << n << " integers: ";
    for (int i = 0; i < n; i++) {
        cin >> nums[i];
    }

    cout << "Enter target sum: ";
    cin >> target;

    // Run brute force subset sum search
    vector<int> result = subsetSumBruteForce(nums, target);

    // Display output
    if (!result.empty()) {
        cout << "Subset found that sums to " << target << ": ";
        for (int x : result) cout << x << " ";
        cout << endl;

    } else {
        cout << "No subset found that sums to " << target << endl;
    }

    return 0;
}
```

# 6. Time and Space Complexity

### Time Complexity:
### $O(n \cdot 2^n)$

Explanation:

- There are $2^n$ subsets.
- Each subset may require checking up to **n** elements.

This results in exponential-time behavior.

### Space Complexity:

**O(n)**

Space usage comes from:

- The temporary vector storing the subset (size ≤ n)
- The input array

No additional exponential memory is required.

# 7. Limitations of Brute Force

1. **Exponential Time Growth**
   When **n > 30**, the number of subsets (more than 33 million) becomes computationally prohibitive.
2. **No Pruning**
   The algorithm inspects all subsets even when partial sums already exceed the target or cannot reach it.
3. **Unsuitable for Large or Real-Time Problems**
   The runtime is too slow for practical large-scale applications.
4. **Redundant Exploration**
   Similar or structured subsets are re-evaluated without optimization.

# 8. Potential Improvements

### 1) Dynamic Programming

- Builds a table of reachable sums
- Runs in **O(n · T)**
- Efficient when **T** is not too large

### 2) Backtracking With Pruning

- Stops exploring as soon as a branch cannot reach the target
- Much more efficient in practice

### 3) Meet-in-the-Middle Algorithm

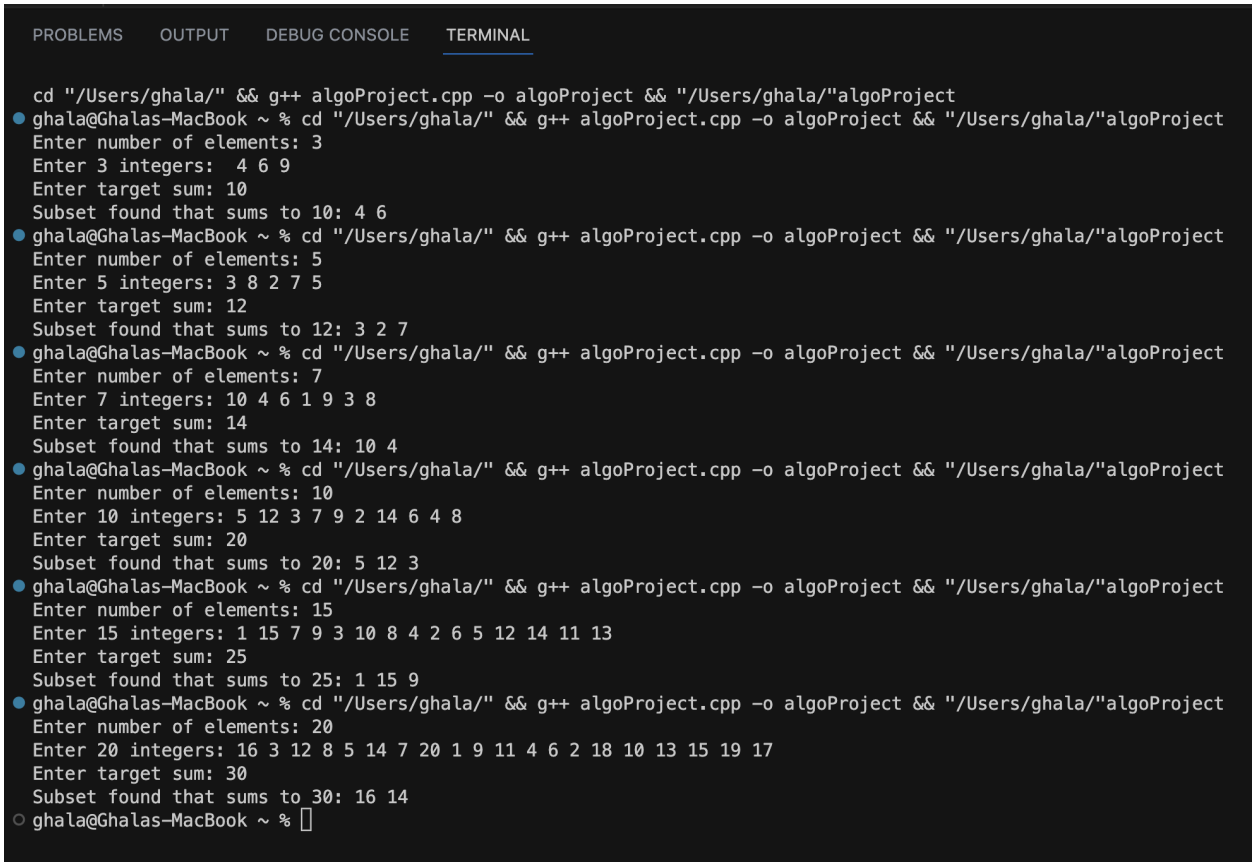
- Splits the set into two halves
- Computes all subset sums for each half
- Reduces runtime from **$O(2^n)$** to **$O(2^{n/2})$**
- Effective for n up to about 40

# 9. Test Cases and Outputs

### Test Cases — Subset Exists:

1. **Input:** 4 6 9, **Target = 10**
   **Output:** Subset found that sums to 10: **4 6**
2. **Input:** 3 8 2 7 5, **Target = 12**
   **Output:** Subset found that sums to 12: **3 2 7**
3. **Input:** 10 4 6 1 9 3 8, **Target = 14**
   **Output:** Subset found that sums to 14: **10 4**
4. **Input:** 5 12 3 7 9 2 14 6 4 8, **Target = 20**
   **Output:** Subset found that sums to 20: **5 12 3**
5. **Input:** 1 15 7 9 3 10 8 4 2 6 5 12 14 11 13, **Target = 25**
   **Output:** Subset found that sums to 25: **1 15 9**
6. **Input:** 16 3 12 8 5 14 7 20 1 9 11 4 6 2 18 10 13 15 19 17, **Target = 30**
   **Output:** Subset found that sums to 30: **16 14**

### Outputs — Subset Exists:




## Test Cases — No Matching Subset

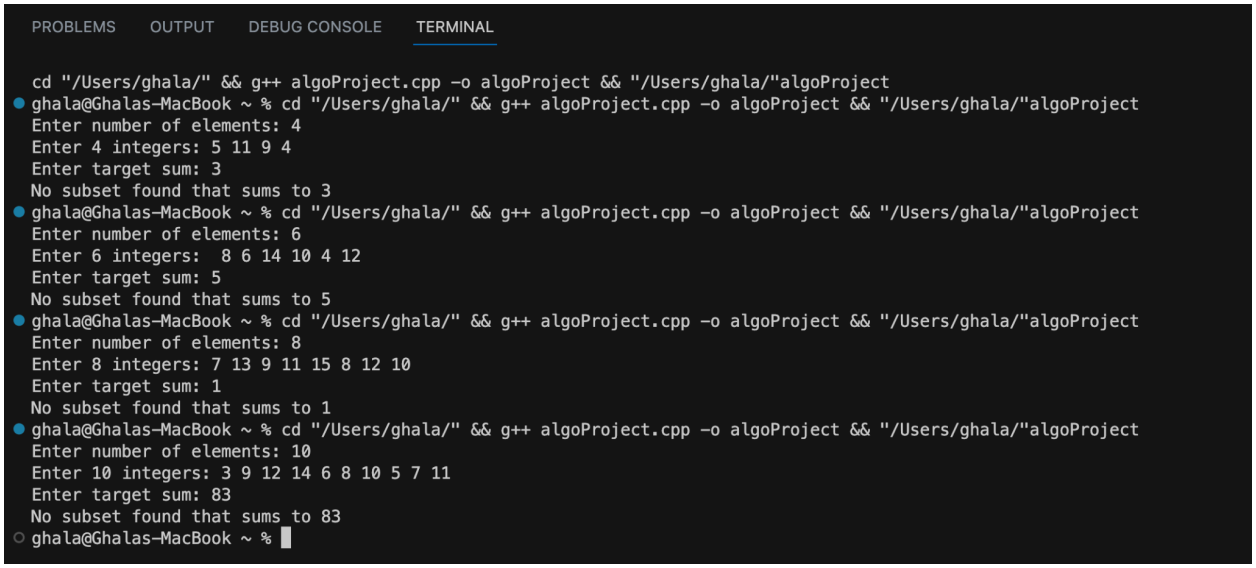
7. **Input:** 5 11 9 4, **Target = 3**
   **Output:** No subset found that sums to 3

8. **Input:** 8 6 14 10 4 12, **Target = 5**
   **Output:** No subset found that sums to 5

9. **Input:** 7 13 9 11 15 8 12 10, **Target = 1**
   **Output:** No subset found that sums to 1

10. **Input:** 3 9 12 14 6 8 10 5 7 11, **Target = 83**
    **Output:** No subset found that sums to 83

## Outputs — No Matching Subset:




# 10. Test Case Summary Table

| Test Case | Input Array | Target | Expected Result | Actual Output |
|---|---|---|---|---|
| **1** | 4 6 9 | 10 | Subset exists | 4 6 |
| **2** | 3 8 2 7 5 | 12 | Subset exists | 3 2 7 |
| **3** | 10 4 6 1 9 3 8 | 14 | Subset exists | 10 4 |
| **4** | 5 12 3 7 9 2 14 6 4 8 | 20 | Subset exists | 5 12 3 |
| **5** | 1 15 7 9 3 10 8 4 2 6 5 12 14 11 13 | 25 | Subset exists | 1 15 9 |
| **6** | 16 3 12 8 5 14 7 20 1 9 11 4 6 2 18 10 13 15 19 17 | 30 | Subset exists | 16 14 |
| **7** | 5 11 9 4 | 3 | No subset exists | No subset found |

| | | | | |
|---|---|---|---|---|
| **8** | 8 6 14 10 4 12 | 5 | No subset exists | No subset found |
| **9** | 7 13 9 11 15 8 12 10 | 1 | No subset exists | No subset found |
| **10** | 3 9 12 14 6 8 10 5 7 11 | 83 | No subset exists | No subset found |

# 11. Conclusion

This project implemented a brute force solution to the Subset Sum Problem using bitmasking to enumerate all possible subsets and check whether any of them matches a target sum. The algorithm consistently produced correct results in all test cases, either identifying a valid subset or confirming that none exists. However, because the number of subsets grows exponentially with the input size, the approach is only practical for small sets, which is why the final program includes a limit on the number of elements. Although more advanced methods such as dynamic programming, backtracking with pruning, or meet-in-the-middle are far more efficient for larger inputs, the brute force method remains valuable for demonstrating exhaustive search and the characteristics of NP-complete problems.